Operating System Practical Solve Slips Sem VI 2

Computer Science (Karnavati University)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 1

Q.1) Write a C Menu driven Program to implement following functionality

a) Accept Available
b) Display Allocation, Max
c) Display the contents of need matrix
d) Display Available

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	2	3	2	9	7	5	3	3	2
P1	4	0	0	5	2	2			
P2	5	0	4	1	0	4			
P3	4	3	3	4	4	4			
P4	2	2	4	6	5	5			

```
#include<stdio.>
 #include<stdlib.h>
 int allocation[20][20],max[20][20],available[20],need[20][20],safe[10],s=0; int
finish[10],work[10],cnt=0,flag=0,temp=0;
 int p,r,i,j,ch,index,req[10];
 void check()
 {
 temp=0;
s=0;
 for(i=0;i<p;i++) //Calculate need=max-allocation
for(j=0;j<r;j++)  need[i][j]=max[i][j]-allocation[i][j];
printf("\nAllocation Table is:\n");  for(i=0;i<p;i++)
 {
 for(j=0;j<r;j++)
 printf("%d\t",allocation[i][j]);
printf("\n");  }
```

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
 printf("\nNeed Table is:\n");
 for(i=0;i<p;i++)
 {
 for(j=0;j<r;j++)
printf("%d\t",need[i][j]);  printf("\n");
 }
 for(i=0;i<p;i++)
finish[i]=0;  for(i=0;i<r;i++)
work[i]=available[i];
 while(temp<2)
 {
 for(i=0;i<p;i++)
 {
 for(j=0;j<r;j++)
 {
 if(finish[i]==0 && need[i][j]<=work[j])
flag=1;  else
 {
 flag=0;
break;
 }
 }
 if(flag==1)
 {
 for(j=0;j<r;j++)
work[j]=work[j]+allocation[i][j];
 finish[i]=1;
safe[s++]=i;
 }
 }
 temp++;
 }
 flag=0;

 for(i=0;i<p;i++)
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
 {
 if(finish[i]==0)
 {
 flag=1;
break;
 }
 }
 if(flag==1)
 {
 printf("\nSystem is in Deadlock state");
 }
 else
 {
 printf("\nSystem is in Safe state");
printf("\nSafe Sequence is:"); for(i=0;i<p;i++)
 printf("P%d\t",safe[i]);
 }
 }
 main()
 {
 printf("\n~~~~~~~~~~~~~~~~~~~~~~~BANKER&apos;S
ALGORITHM~~~~~~~~~~~~~~~~~~~~~~~~~~");
 printf("\n\nEnter the no of resources and processes: ");
 scanf("%d%d",&r,&p);  printf("\nEnter the
Allocation Table:\n");  for(i=0;i<p;i++)
//Accept the allocation table
 for(j=0;j<r;j++)
scanf("%d",&allocation[i][j]);  printf("\nEnter
the Max Table:\n");  for(i=0;i<p;i++)
//Accept the max table
 for(j=0;j<r;j++)  scanf("%d",&max[i][j]);
printf("\nEnter the vector Available :");
for(i=0;i<r;i++)  scanf("%d",&available[i]);
check();  printf("\nDo U want to add new
request:(0/1)");  scanf("%d",&ch);  if(ch==0)
exit(1);
 printf("\nEnter the process no:");
scanf("%d",&index);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
printf("\nEnter the request:");
for(i=0;i<r;i++)
scanf("%d",&req[i]);  flag=0;
for(i=0;i<r;i++)
if(req[i]<=need[index][i])  flag=1;
else  flag=0;  if(flag==0)
 {
 printf("\nRequest can not be satisfied...");
 exit(1);
 }
 for(i=0;i<r;i++)
if(req[i]<=available[i])
flag=1;  else  flag=0;
if(flag==0)
 {
 printf("\nRequest can not be satisfied...");
exit(1);
 }
 for(i=0;i<r;i++)
 {
 allocation[index][i]=allocation[index][i]+req[i];  available[i]=available[i]-
req[i];
 }

 check();
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

}

[15]

Q.2 Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

55, 58, 39, 18, 90, 160, 150, 38, 184

Start Head Position: 50

```
#include<math.h>
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int i,n,req[50],mov=0,cp;
   printf("enter the current position\n");
scanf("%d",&cp);
   printf("enter the number of requests\n");
scanf("%d",&n);
   printf("enter the request order\n");
for(i=0;i<n;i++)
   {
      scanf("%d",&req[i]);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
    }
    mov=mov+abs(cp-req[0]); // abs is used to calculate the absolute
value     printf("%d -> %d",cp,req[0]);     for(i=1;i<n;i++)
    {
        mov=mov+abs(req[i]-req[i-1]);
printf(" -> %d",req[i]);
    }
    printf("\n");
    printf("total head movement = %d\n",mov);
}
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Slip 2

Q.1 Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h> void

main()

{

int f[50], p,i, st, len, j, c, k, a;

clrscr(); for(i=0;i<50;i++)

f[i]=0;

printf("Enter how many blocks already allocated: ");

scanf("%d",&p);

printf("Enter blocks already allocated: ");

for(i=0;i<p;i++)

{
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
scanf("%d",&a); f[a]=1;
}
x: printf("Enter index starting block and length: ");
scanf("%d%d", &st,&len); k=len; if(f[st]==0)
{
for(j=st;j<(st+k);j++)
{ if(f[j]==0)
{ f[j]=1; printf("%d--------
>%d\n",j,f[j]);
} else
{
printf("%d Block is already allocated \n",j); k++;
}
}
} else
printf("%d starting block is already allocated \n",st);
printf("Do you want to enter more file(Yes - 1/No - 0)");
scanf("%d", &c); if(c==1) goto x; else exit(0); getch();
}
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

[15]

Q.2 Write an MPI program to calculate sum of randomly generated 1000 numbers (stored in array) on a cluster

```
#include <stdio.h>

#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000

int main(int argc, char *argv[]) {
   int rank, size;
   int local_array[ARRAY_SIZE / 2];
   int global_sum = 0, local_sum = 0;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

for (int i = 0; i < ARRAY_SIZE / 2; i++) {
      local_array[i] = rand() % 1000; // Random numbers between 0 and 999
local_sum += local_array[i];
   }

   MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

if (rank == 0) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        printf("Global sum: %d\n", global_sum);
    }

    MPI_Finalize();
return 0;
}
```

SLIP3

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

a) Calculate and display the content of need matrix?
b) Is the system in safe state? If display the safe sequence.

```
#include<stdio.h>
#include<stdbool.h>

#define P 5 // Number of processes
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
#define R 4 // Number of resources

void calculateNeed(int need[P][R], int max[P][R], int alloc[P][R]) {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}

isSafe(int processes[], int avail[], int max[][R], int alloc[][R]) {
    int need[P][R];
    calculateNeed(need, max, alloc);

    bool finish[P] = {0};
    int safeSeq[P];
    int work[R];
    for (int i = 0; i < R; i++) {
        work[i] = avail[i];
    }

    int count = 0;
    while (count < P) {
        bool found = false;
        for (int p = 0; p < P; p++) {
            if (!finish[p]) {
                int j;
                for (j = 0; j < R; j++) {
                    if (need[p][j] > work[j])
                        break;
                }
                if (j == R) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
                for (int k = 0; k < R; k++)
                    work[k] += alloc[p][k];

                safeSeq[count++] = p;
                finish[p] = 1;
                found = true;
            }
        }
    }
    if (!found) {
        printf("System is not in safe state\n");
        return false;
    }
}

printf("System is in safe state.\nSafe sequence is: ");
for (int i = 0; i < P; i++) {
    printf("P%d ", safeSeq[i]);
}
printf("\n");
return true;
}

int main() {
    int processes[] = {0, 1, 2, 3, 4};

    int alloc[P][R] = {
        {0, 0, 1, 2},
        {1, 0, 0, 0},
        {1, 3, 5, 4},
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        {0, 6, 3, 2},
        {0, 0, 1, 4}
    };

    int max[P][R] = {
        {0, 0, 1, 2},
        {1, 7, 5, 0},
        {2, 3, 5, 6},
        {0, 6, 5, 2},
        {0, 6, 5, 6}
    };

    int avail[] = {1, 5, 2, 0};

    isSafe(processes, avail, max, alloc);

    return 0;
}
```

[15]

Q.2 Write an MPI program to calculate sum and average of randomly generated 1000 numbers (stored in array) on a cluster [15]

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000

int main(int argc, char *argv[]) {
    int rank, size;
    int local_array[ARRAY_SIZE / 2];
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   int global_sum = 0, local_sum = 0;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   for (int i = 0; i < ARRAY_SIZE / 2; i++)
{
       local_array[i] = rand() % 1000;
        local_sum += local_array[i];
   }

   MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

if (rank == 0) {

       printf("Global sum: %d\n", global_sum);
   }

   MPI_Finalize();
return 0;
}
```

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 4

Q.1 Implement the Menu driven Banker's algorithm for accepting Allocation, Max from user.

a) Accept Available
b) Display Allocation, Max

c) Find Need and display It,
d) Display Available

Consider the system with 3 resources types A,B, and C with 7,2,6 instances respectively. Consider the following snapshot:

Process	Allocation			Request		
	A	B	C	A	B	C
P0	0	1	0	0	0	0
P1	4	0	0	5	2	2
P2	5	0	4	1	0	4
P3	4	3	3	4	4	4
P4	2	2	4	6	5	5

[15]

```
#include <stdio.h>

#define NUM_RESOURCES 3
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
#define NUM_PROCESSES 5

int available[NUM_RESOURCES] = {7, 2, 6}; int
allocation[NUM_PROCESSES][NUM_RESOURCES]; int
max_claim[NUM_PROCESSES][NUM_RESOURCES];

void calculate_need(int need[NUM_PROCESSES][NUM_RESOURCES])
{     for (int i = 0; i < NUM_PROCESSES; i++) {         for (int j = 0; j <
NUM_RESOURCES; j++) {             need[i][j] = max_claim[i][j] -
allocation[i][j];
        }
    }
}


// Function to display allocation matrix void
display_allocation() {     printf("Allocation
Matrix:\n");     for (int i = 0; i <
NUM_PROCESSES; i++) {
        for (int j = 0; j < NUM_RESOURCES; j++) {
printf("%d ", allocation[i][j]);
        }
        printf("\n");
    }
}

// Function to display max claim matrix void
display_max_claim() {     printf("Max Claim
Matrix:\n");     for (int i = 0; i <
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
NUM_PROCESSES; i++) {        for (int j = 0; j <
NUM_RESOURCES; j++) {
          printf("%d ", max_claim[i][j]);
       }
       printf("\n");
    }
}

void display_need(int need[NUM_PROCESSES][NUM_RESOURCES])
{     printf("Need Matrix:\n");     for (int i = 0; i < NUM_PROCESSES;
i++) {
       for (int j = 0; j < NUM_RESOURCES; j++) {
printf("%d ", need[i][j]);        }

       printf("\n");
    }
}

// Function to display available resources void
display_available() {     printf("Available
Resources: ");     for (int i = 0; i <
NUM_RESOURCES; i++) {        printf("%d ",
available[i]);
    }
   printf("\n");
}

int main() {
char choice;
   int need[NUM_PROCESSES][NUM_RESOURCES];

   while (1) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        printf("\nBanker's Algorithm Menu:\n");
printf("a) Accept Available\n");
printf("b) Display Allocation, Max\n");
printf("c) Find Need and display It\n");
printf("d) Display Available\n");
printf("e) Exit\n");        printf("Enter your
choice: ");
        scanf(" %c", &choice);

        switch (choice) {
case 'a':
                printf("Enter available resources for A, B, and C: ");
scanf("%d %d %d", &available[0], &available[1], &available[2]);
break;            case 'b':
                printf("Enter allocation matrix:\n");
for (int i = 0; i < NUM_PROCESSES; i++) {
                    printf("For process %d: ", i);
                    scanf("%d %d %d", &allocation[i][0], &allocation[i][1], &allocation[i][2]);
                }
                printf("Enter max claim matrix:\n");
for (int i = 0; i < NUM_PROCESSES; i++) {
                    printf("For process %d: ", i);
                    scanf("%d      %d     %d",  &max_claim[i][0],          &max_claim[i][1],
      &max_claim[i][2]);
                }
break;
case 'c':
                calculate_need(need);
display_need(need);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            break;          case
 'd':
 display_available();
            break;
 case 'e':
            printf("Exiting...\n");
            return 0;
 default:
            printf("Invalid choice. Please try again.\n");
        }
    }


   return 0;
 }
```

Q.2 Write a simulation program for disk scheduling using SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments. 86, 147, 91, 170, 95, 130, 102, 70 Starting Head position= 125 Direction: Left

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_REQUESTS 1000

// Function to sort disk requests in ascending
order
void sort_requests(int requests[], int
num_requests) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    int i, j, temp;     for (i = 0; i <
num_requests - 1; i++)
{         for (j = 0; j < num_requests - i -
1; j++) {
            if (requests[j] > requests[j + 1])
{                temp = requests[j];
requests[j] = requests[j + 1];
               requests[j + 1] = temp;
            }

        }
    }
}

// Function to simulate SCAN disk scheduling
algorithm
void scan(int requests[], int num_requests, int start,
char direction)
{     int total_head_movements = 0;
int current_head_position = start;
    int index;

    // Sort requests
sort_requests(requests, num_requests);

    // Find the index where current head position
would fit
    for       (index        =       0;       index <
num_requests; index++) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        if (requests[index] >=
current_head_position) {
break;
        }
    }

    // If direction is left, reverse the
sorted requests    if (direction == 'L') {

        for (int i = 0, j = num_requests - 1;
i < j; i++, j--) {                int temp =
requests[i];                    requests[i] =
requests[j];            requests[j] = temp;
        }
    }

    // Scan towards lower track    for (int
i = index; i < num_requests; i++) {
printf("%d ", requests[i]);
        total_head_movements += abs(requests[i] -
current_head_position);
        current_head_position = requests[i];
    }

    // If direction is left and there are
requests to the left of current head
position    if (direction == 'L' && index
> 0) {
        printf("%d ", 0);
        total_head_movements +=
current_head_position;
current_head_position = 0;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        // Scan towards higher track
    for (int i = index - 1; i >= 0; i--) {
    printf("%d ", requests[i]);
    total_head_movements        +=
    abs(requests[i]   -
    current_head_position);
            current_head_position          =
    requests[i];
        }
      }

      printf("\nTotal head movements: %d\n",
    total_head_movements);
    }

    int main() {
      int
    requests[MAX_REQUESTS];
    int num_requests, start;     char
    direction;

      // Input total number of disk blocks,
    disk request string, and current head
    position
      printf("Enter the total number of disk
    blocks: ");
   scanf("%d", &num_requests
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 6

Q.1 Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

[15]

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>

#define MAX_BLOCKS 100

bool bit_vector[MAX_BLOCKS]; int
directory[MAX_BLOCKS];
int num_blocks;

// Function to initialize the bit vector and directory void
initialize() {
   for (int i = 0; i < num_blocks; i++) {
bit_vector[i] = false;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        directory[i] = -1;
    }
}

// Function to display the bit vector
void show_bit_vector() {     printf("Bit
Vector:\n");     for (int i = 0; i <
num_blocks; i++) {
        printf("%d ", bit_vector[i]);
    }
    printf("\n");
}

// Function to create a new file
void create_new_file(int file_index, int file_size) {
int num_allocated = 0;
    int current_block = rand() % num_blocks; // Start from a random block

    while (num_allocated < file_size) {         if
(!bit_vector[current_block]) {
bit_vector[current_block] = true;
directory[current_block] = file_index;
num_allocated++;
        }
        current_block = (current_block + 1) % num_blocks; // Move to the next block
    }
}

// Function to display the directory
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
void show_directory() {
printf("Directory:\n");     for (int i = 0;
i < num_blocks; i++) {         if
(directory[i] != -1) {
          printf("Block %d -> File %d\n", i, directory[i]);
       }
    }
}

int main() {
   srand(time(NULL)); // Seed random number generator

   // Input the number of blocks on the disk
printf("Enter the number of blocks on the disk: ");
   scanf("%d", &num_blocks);

   initialize(); // Initialize bit vector and directory

   char choice;     int
file_index, file_size;

   // Menu-driven program     while
(1) {         printf("\nMenu:\n");
printf("1. Show Bit Vector\n");
printf("2. Create New File\n");
printf("3. Show Directory\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf(" %c", &choice);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
      switch (choice) {
case '1':
            show_bit_vector();
            break;
case '2':
            printf("Enter the file index: ");
scanf("%d", &file_index);
            printf("Enter the file size (number of blocks): ");
scanf("%d", &file_size);
            create_new_file(file_index, file_size);
break;          case '3':
            show_directory();
            break;
case '4':
            printf("Exiting...\n");
            return 0;
default:
            printf("Invalid choice. Please try again.\n");
      }
   }

   return 0;
  }
```

Q.2 Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments..

80, 150, 60,135, 40, 35, 170 Starting Head Position: 70 Direction: Right

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
#include <stdio.h>

#include <stdlib.h>


#define MAX_REQUESTS 1000


// Function to sort disk requests in ascending order void

sort_requests(int requests[], int num_requests) {     int

i, j, temp;

    for (i = 0; i < num_requests - 1; i++) {

for (j = 0; j < num_requests - i - 1; j++) {

if (requests[j] > requests[j + 1]) {

temp = requests[j];                requests[j] =

requests[j + 1];                requests[j + 1] =

temp;

            }

        }

    }

}


// Function to simulate C-SCAN disk scheduling algorithm

void cscan(int requests[], int num_requests, int start) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
int total_head_movements = 0;     int
current_head_position = start;

    // Sort requests
    sort_requests(requests, num_requests);

    // Find the index where current head position would fit
int index;
    for (index = 0; index < num_requests; index++) {
if (requests[index] >= current_head_position) {
break;
        }
    }

    // Scan towards higher track
    for (int i = index; i < num_requests; i++) {
printf("%d ", requests[i]);
        total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    }

    // If there are requests to the left of current head position, wrap around to the beginning
if (index > 0) {
        total_head_movements += abs(current_head_position);
current_head_position = 0;        printf("%d ", 0);
    }

    // Scan towards higher track again if not at the end
for (int i = 0; i < index; i++) {        printf("%d ",
requests[i]);
        total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
    }

    printf("\nTotal head movements: %d\n", total_head_movements); }

int main() {
    int requests[MAX_REQUESTS];
int num_requests, start;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)
CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   // Input total number of disk blocks, disk request string, and current head position
printf("Enter the total number of disk blocks: ");     scanf("%d", &num_requests);


   printf("Enter the disk request string:\n");
for (int i = 0; i < num_requests; i++) {
scanf("%d", &requests[i]);
   }


   printf("Enter the starting head position: ");
scanf("%d", &start);


   // Perform C-SCAN disk scheduling algorithm
printf("Order of service:\n");     cscan(requests,
num_requests, start);


   return 0;
  }
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 8

Q.1 Write a program to simulate Contiguous file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned above and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>


#define MAX_BLOCKS 100

bool bit_vector[MAX_BLOCKS]; int
directory[MAX_BLOCKS];
int num_blocks;

// Function to initialize the bit vector and directory void
initialize() {
   for (int i = 0; i < num_blocks; i++) {
      bit_vector[i] = false;
directory[i] = -1;
   }
}
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
// Function to display the bit vector
void show_bit_vector() {     printf("Bit
Vector:\n");     for (int i = 0; i <
num_blocks; i++) {
        printf("%d ", bit_vector[i]);
    }
    printf("\n");
}

// Function to create a new file
void create_new_file(int file_index, int file_size) {
int num_allocated = 0;
    int current_block = rand() % num_blocks; // Start from a random block

    while (num_allocated < file_size) {

        if (!bit_vector[current_block]) {
bit_vector[current_block] = true;
directory[current_block] = file_index;
num_allocated++;
        }
        current_block = (current_block + 1) % num_blocks; // Move to the next block
    }
}

// Function to display the directory
void show_directory() {
printf("Directory:\n");
    for (int i = 0; i < num_blocks; i++) {
        if (directory[i] != -1) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            printf("Block %d -> File %d\n", i, directory[i]);
        }
    }
}

int main() {
    srand(time(NULL)); // Seed random number generator

    // Input the number of blocks on the disk
printf("Enter the number of blocks on the disk: ");
    scanf("%d", &num_blocks);

    initialize(); // Initialize bit vector and directory

    char choice;     int
file_index, file_size;     //
Menu-driven program
while (1) {
printf("\nMenu:\n");
printf("1. Show Bit
Vector\n");        printf("2.
Create New File\n");
printf("3. Show
Directory\n");
printf("4. Exit\n");
printf("Enter your choice:
");
        scanf(" %c", &choice);

        switch (choice) {
case '1':
                show_bit_vector();
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            break;
case '2':
            printf("Enter the file index: ");
scanf("%d", &file_index);
            printf("Enter the file size (number of blocks): ");
scanf("%d", &file_size);
create_new_file(file_index, file_size);              break;
case '3':
            show_directory();
            break;          case
'4':
printf("Exiting...\n");
            return 0;
default:
printf("Invalid choice.
Please try again.\n");
        }
    }

   return 0;
  }
```

[15]

Q.2 Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

186, 89, 44, 70, 102, 22, 51, 124

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Start Head Position: 70

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define MAX_REQUESTS 1000

// Function to simulate SSTF disk scheduling algorithm void
sstf(int requests[], int num_requests, int start) {
   int total_head_movements = 0;
   int current_head_position = start;

   // Initialize an array to keep track of whether each request has been serviced
int serviced[MAX_REQUESTS];
   for (int i = 0; i < num_requests; i++) {
serviced[i] = 0;

   }

   // Service requests     printf("Order of
service:\n");     for (int i = 0; i <
num_requests; i++) {         int
min_distance = INT_MAX;
       int next_request_index = -1;

       // Find the request with the shortest seek time
for (int j = 0; j < num_requests; j++) {
          if (!serviced[j]) {
             int distance = abs(requests[j] - current_head_position);
if (distance < min_distance) {                    min_distance =
distance;
                next_request_index = j;
             }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        }
    }

    // Mark the request as serviced
serviced[next_request_index] = 1;

    // Update total head movements
    total_head_movements += min_distance;

    // Move the head to the next request
    current_head_position = requests[next_request_index];

    // Print the serviced request

    printf("%d ", requests[next_request_index]);
  }

  printf("\nTotal head movements: %d\n", total_head_movements); }

int main() {
  int requests[MAX_REQUESTS];
int num_requests, start;

  // Input total number of disk blocks, disk request string, and current head position
printf("Enter the total number of disk blocks: ");
  scanf("%d", &num_requests);

  printf("Enter the disk request string:\n");
for (int i = 0; i < num_requests; i++) {
scanf("%d", &requests[i]);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    }

    printf("Enter the starting head position: ");
scanf("%d", &start);

    // Perform SSTF disk scheduling algorithm
sstf(requests, num_requests, start);

    return 0;
  }
```

[15]

Q.3. Oral/Viva [05] SLIP 10

Q.1 Write an MPI program to calculate sum and average of randomly generated 1000 numbers (stored in array) on a cluster

```
#include <stdio.h>

#include <stdlib.h>

#include <mpi.h>


#define ARRAY_SIZE 1000


int main(int argc, char *argv[]) {     int rank, size;

int local_array[ARRAY_SIZE / 2];     int

global_sum = 0, local_sum = 0;     double
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
global_average = 0.0, local_average = 0.0;

MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);

   MPI_Comm_size(MPI_COMM_WORLD, &size);


   // Seed random number generator differently on each process

srand(rank);


   // Generate local random numbers     for (int i = 0; i < ARRAY_SIZE / 2;

i++) {         local_array[i] = rand() % 1000; // Random numbers between 0

and 999         local_sum += local_array[i];

   }


   // Sum local sums across all processes

   MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);


   // Calculate global average     if (rank == 0) {

global_average = (double)global_sum / ARRAY_SIZE;     }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   // Broadcast global average to all processes

   MPI_Bcast(&global_average, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);


   // Print sum and average only from the root process

if (rank == 0) {        printf("Global sum: %d\n",

global_sum);        printf("Global average: %.2f\n",

global_average);

    }


   MPI_Finalize();

return 0;

  }
```

[15]

Q.2 Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

33, 99, 142, 52, 197, 79, 46, 65

Start Head Position: 72
Direction: Left

```
#include <stdio.h>
#include <stdlib.h>
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
#define MAX_REQUESTS 1000

// Function to sort disk requests in ascending order
void sort_requests(int requests[], int num_requests) {
int i, j, temp;
   for (i = 0; i < num_requests - 1; i++) {
for (j = 0; j < num_requests - i - 1; j++) {
if (requests[j] > requests[j + 1]) {
temp = requests[j];              requests[j] =
requests[j + 1];
            requests[j + 1] = temp;
         }
      }
   }
}

// Function to simulate C-SCAN disk scheduling algorithm
void cscan(int requests[], int num_requests, int start) {
int total_head_movements = 0;
   int current_head_position = start;

   // Sort requests
   sort_requests(requests, num_requests);


   // Find the index where current head position would fit
int index;
   for (index = 0; index < num_requests; index++) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        if (requests[index] >= current_head_position) {
break;
        }
    }

    // Scan towards higher track
    for (int i = index; i < num_requests; i++) {
        printf("%d ", requests[i]);
        total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
    }

    // If there are requests to the left of current head position, wrap around to the beginning
if (index > 0) {
        total_head_movements += abs(current_head_position);
current_head_position = 0;
        printf("%d ", 0);
    }

    // Scan towards higher track again if not at the end
    for (int i = 0; i < index; i++) {
printf("%d ", requests[i]);
        total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
    }


    printf("\nTotal head movements: %d\n", total_head_movements); }

int main() {
    int requests[MAX_REQUESTS];
int num_requests, start;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   // Input total number of disk blocks, disk request string, and current head position
printf("Enter the total number of disk blocks: ");
   scanf("%d", &num_requests);

   printf("Enter the disk request string:\n");
for (int i = 0; i < num_requests; i++) {
scanf("%d", &requests[i]);
   }

   printf("Enter the starting head position: ");
scanf("%d", &start);

   // Perform C-SCAN disk scheduling algorithm
   printf("Order of service:\n");
cscan(requests, num_requests, start);

   return 0;
  }
```
[15]

Q.3. Oral/Viva [05] SLIP 11

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. the following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Implement the following Menu.

a) Accept Available
b) Display Allocation, Max
c) Display the contents of need matrix
d) Display Available [15]

```
#include <stdio.h>

#define MAX_PROCESSES 5
#define MAX_RESOURCES 3

int allocation[MAX_PROCESSES][MAX_RESOURCES] = {
   {0, 1, 0},
   {2, 0, 0},
   {3, 0, 3},
   {2, 1, 1},
   {0, 0, 2}

};

int max[MAX_PROCESSES][MAX_RESOURCES] = {
   {0, 0, 0},
   {2, 0, 2},
   {0, 0, 0},
   {1, 0, 0},
   {0, 0, 2}
};

int need[MAX_PROCESSES][MAX_RESOURCES];
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
int available[MAX_RESOURCES] = {0};

// Function to calculate need matrix void
calculate_need() {     for (int i = 0; i <
MAX_PROCESSES; i++) {         for (int j = 0; j <
MAX_RESOURCES; j++) {             need[i][j] =
max[i][j] - allocation[i][j];
        }
    }
}

// Function to accept available resources void
accept_available() {     printf("Enter the available
resources for each type:\n");     for (int i = 0; i <
MAX_RESOURCES; i++) {         printf("Resource type
%c: ", 'A' + i);
        scanf("%d", &available[i]);
    }
}


// Function to display allocation and max matrices
void display_allocation_max() {
printf("Allocation Matrix:\n");     for (int i = 0; i <
MAX_PROCESSES; i++) {         printf("P%d: ",
i);
        for (int j = 0; j < MAX_RESOURCES; j++) {
printf("%d ", allocation[i][j]);
        }
printf("\n");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    }

    printf("\nMax Matrix:\n");     for (int i = 0; i
< MAX_PROCESSES; i++) {
printf("P%d: ", i);
        for (int j = 0; j < MAX_RESOURCES; j++) {
printf("%d ", max[i][j]);
        }
printf("\n");
    }
}

// Function to display need matrix void
display_need() {     printf("Need Matrix:\n");
for (int i = 0; i < MAX_PROCESSES; i++) {
printf("P%d: ", i);
        for (int j = 0; j < MAX_RESOURCES; j++) {
printf("%d ", need[i][j]);

        }
printf("\n");
    }
}

// Function to display available resources void
display_available() {     printf("Available Resources:\n");
for (int i = 0; i < MAX_RESOURCES; i++) {
printf("Resource type %c: %d\n", 'A' + i, available[i]);
    }
}

int main() {
    calculate_need();
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   char choice;     while (1) {         printf("\nMenu:\n");
printf("a) Accept Available\n");         printf("b)
Display Allocation, Max\n");         printf("c) Display
the contents of need matrix\n");         printf("d) Display
Available\n");         printf("e) Exit\n");
printf("Enter your choice: ");
       scanf(" %c", &choice);

       switch (choice) {
case 'a':
              accept_available();

break;            case 'b':
              display_allocation_max();
              break;
case 'c':
display_need();
              break;            case
'd':
display_available();
              break;
case 'e':
              printf("Exiting...\n");
              return 0;
default:
              printf("Invalid choice. Please try again.\n");
       }
   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   return 0;
  }
```

Q.2 Write an MPI program to find the min number from randomly generated 1000 numbers (stored in array) on a cluster (Hint: Use MPI_Reduce)

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000


int main(int argc, char *argv[]) {
   int rank, size;
   int local_array[ARRAY_SIZE / 2];
int global_sum = 0, local_sum = 0;
   double global_average = 0.0, local_average = 0.0;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   // Seed random number generator differently on each process
srand(rank);

   // Generate local random numbers     for
(int i = 0; i < ARRAY_SIZE / 2; i++) {
       local_array[i] = rand() % 1000; // Random numbers between 0 and 999
local_sum += local_array[i];
   }

   // Sum local sums across all processes
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

   // Calculate global average
if (rank == 0) {
       global_average = (double)global_sum / ARRAY_SIZE;
   }

   // Broadcast global average to all processes
   MPI_Bcast(&global_average, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);


   // Print sum and average only from the root process
if (rank == 0) {
       printf("Global sum: %d\n", global_sum);
       printf("Global average: %.2f\n", global_average);
   }

   MPI_Finalize();
   return 0;
  }
```

[15]

Q.3. Oral/Viva [05]

SLIP 13

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance.

The following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

a) Calculate and display the content of need matrix?

b) Is the system in safe state? If display the safe sequence. [15]

```
#include <stdio.h>

#define MAX_PROCESSES 5
#define MAX_RESOURCES 3

int allocation[MAX_PROCESSES][MAX_RESOURCES] = {
   {0, 1, 0},
   {2, 0, 0},
   {3, 0, 3},
   {2, 1, 1},
   {0, 0, 2}
};

int max[MAX_PROCESSES][MAX_RESOURCES] = {
   {0, 0, 0},
{2, 0, 2},
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    {0, 0, 0},
    {1, 0, 0},
    {0, 0, 2}
};

int available[MAX_RESOURCES] = {0}; int
need[MAX_PROCESSES][MAX_RESOURCES];

// Function to calculate need matrix void
calculate_need() {

    for (int i = 0; i < MAX_PROCESSES; i++) {
for (int j = 0; j < MAX_RESOURCES; j++) {
need[i][j] = max[i][j] - allocation[i][j];
        }
    }
}

// Function to check if the system is in a safe state
int is_safe_state(int finished[], int work[]) {     int
work_copy[MAX_RESOURCES];
    int i, j, k;
    for (i = 0; i < MAX_RESOURCES; i++) {
        work_copy[i] = work[i];
    }
    int safe_sequence[MAX_PROCESSES];
    int count = 0;

    while (count < MAX_PROCESSES) {
        int found = 0;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
    for (i = 0; i < MAX_PROCESSES; i++) {
        if (!finished[i]) {
            int need_met = 1;
            for (j = 0; j < MAX_RESOURCES; j++) {
if (need[i][j] > work_copy[j]) {
                need_met = 0;
break;
            }
        }            if
(need_met) {
found = 1;
finished[i] = 1;
safe_sequence[count++]
= i;            for (k =
0; k <
MAX_RESOURCES;
k++) {
                work_copy[k] += allocation[i][k];
            }
break;
        }
    }     }
if (!found) {
        return 0; // System is not in a safe state
    }
  }

  // Print safe sequence    printf("Safe
sequence: ");    for (i = 0; i <
MAX_PROCESSES - 1; i++) {
    printf("P%d -> ", safe_sequence[i]);
  }
  printf("P%d\n", safe_sequence[MAX_PROCESSES - 1]);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   return 1; // System is in a safe state
}

// Function to display the content of need matrix
void display_need_matrix() {     printf("Need
Matrix:\n");     for (int i = 0; i <
MAX_PROCESSES; i++) {         printf("P%d:
", i);

       for (int j = 0; j < MAX_RESOURCES; j++) {
printf("%d ", need[i][j]);
       }
       printf("\n");
   }
}

int main() {
calculate_need();

   printf("a) Need Matrix:\n");
   display_need_matrix();

   printf("\nb) ");
   int finished[MAX_PROCESSES] = {0};
   printf(is_safe_state(finished, available) ? "System is in safe state.\n" : "System is not in
safe state.\n");

   return 0;
  }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.2 Write a simulation program for disk scheduling using SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

176, 79, 34, 60, 92, 11, 41, 114 Starting
Head Position: 65
Direction: Left

```
#include <stdio.h>
#include <stdlib.h>

// Function to sort disk requests in ascending order
void sort_requests(int requests[], int num_requests) {
int i, j, temp;
   for (i = 0; i < num_requests - 1; i++) {
for (j = 0; j < num_requests - i - 1; j++) {
if (requests[j] > requests[j + 1]) {
temp = requests[j];              requests[j] =
requests[j + 1];
            requests[j + 1] = temp;
         }
      }
   }
}

// Function to simulate SCAN disk scheduling algorithm void
scan(int requests[], int num_requests, int start, char direction) {
int total_head_movements = 0;     int current_head_position = start;

   // Sort requests
   sort_requests(requests, num_requests);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   // Find the index where current head position would fit
int index;
   for (index = 0; index < num_requests; index++) {
      if (requests[index] >= current_head_position) {
break;
      }

   }

   // Determine direction of movement
int i;
   if (direction == 'L') {        //
Scan towards lower track        for
(i = index - 1; i >= 0; i--) {
printf("%d ", requests[i]);
         total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
      }
      // Go to the lower boundary if needed
if (index < num_requests) {
         total_head_movements += abs(0 - current_head_position);
current_head_position = 0;
      }
      // Scan towards higher track        for (i
= index; i < num_requests; i++) {
         printf("%d ", requests[i]);
         total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
      }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   } else if (direction == 'R') {         // Scan
towards higher track         for (i = index; i <
num_requests; i++) {
          printf("%d ", requests[i]);
          total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
       }

       // Go to the higher boundary if needed
if (index > 0) {
          total_head_movements += abs(199 - current_head_position); // Assuming disk size
is 200
          current_head_position = 199; // Assuming disk size is 200
       }
       // Scan towards lower track
for (i = index - 1; i >= 0; i--) {
printf("%d ", requests[i]);
          total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
       }
   }

   printf("\nTotal head movements: %d\n", total_head_movements); }

int main() {     int requests[] = {176, 79, 34, 60, 92, 11, 41,
114};     int num_requests = sizeof(requests) /
sizeof(requests[0]);     int start = 65;     char direction = 'L';

   printf("Order of service:\n");
   scan(requests, num_requests, start, direction);

   return 0;
  }
```

[15]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.3. Oral/Viva [05]

SLIP 14

Q.1 Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Show Directory
- Delete File
- Exit
-

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_BLOCKS 1000
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

int disk_blocks[MAX_BLOCKS] = {0}; // 0 indicates free block, 1 indicates allocated block

// Function to randomly mark some blocks as allocated void random_allocation(int total_blocks) { srand(time(NULL)); // Seed the random number generator int num_allocated = rand() % (total_blocks + 1); // Randomly select the number of allocated blocks

printf("Randomly allocated blocks: "); for (int i = 0; i < num_allocated; i++) { int block = rand() % total_blocks; // Randomly select a block if (disk_blocks[block] == 0) { // If the block is free disk_blocks[block] = 1; // Mark the block as allocated printf("%d ", block);
} else { // If the block is already allocated, try another block
i--;
} }
printf("\n");
}

// Function to display the bit vector void show_bit_vector(int total_blocks) { printf("Bit Vector (0 indicates free block, 1 indicates allocated block):\n"); for

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
(int i = 0; i < total_blocks; i++) {
printf("%d ", disk_blocks[i]);
   }
printf("\n");
}

// Function to display the directory void show_directory(int
total_blocks) {     printf("Directory (Allocated blocks):\n");
printf("Start Block\tLength\n");     int start_block = -1;     int length = 0;
for (int i = 0; i < total_blocks; i++) {         if (disk_blocks[i] == 1) { // If
the block is allocated            if (start_block == -1) { // If this is the start
of a contiguous block                start_block = i;               length = 1;
          } else { // If this block is part of the contiguous block
length++;
          }
       } else { // If the block is free

          if (start_block != -1) { // If the contiguous block has
ended                printf("%d\t\t%d\n", start_block, length);
start_block = -1;                length = 0;
          }
       }
   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    if (start_block != -1) { // If the last contiguous block extends to the end
printf("%d\t\t%d\n", start_block, length);
    }
}

// Function to delete a file (mark its blocks as free) void delete_file(int total_blocks) {
printf("Enter the starting block of the file to delete: ");     int start_block;     scanf("%d",
&start_block);     if (start_block >= 0 && start_block < total_blocks &&
disk_blocks[start_block] == 1)
{         int length =
0;
        // Find the length of the file         for (int i =
start_block; i < total_blocks; i++) {             if
(disk_blocks[i] == 1) {

                length++;
} else {
break;
            }
        }
        // Mark the blocks as free         for (int i = start_block;
i < start_block + length; i++) {             disk_blocks[i] = 0;
        }
        printf("File deleted successfully.\n");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    } else {        printf("Invalid starting block or file not
found.\n");
    }
}

int main() {     int
total_blocks;

    printf("Enter the total number of disk blocks: ");
scanf("%d", &total_blocks);

    random_allocation(total_blocks);


    char choice;     do {
printf("\nMenu:\n");        printf("a)
Show Bit Vector\n");
printf("b) Show Directory\n");
printf("c) Delete File\n");
printf("d) Exit\n");
printf("Enter your choice: ");
scanf(" %c", &choice);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        switch (choice) {
    case 'a':

    show_bit_vector(total_blocks);
    break;          case 'b':

    show_directory(total_blocks);
    break;          case 'c':

    delete_file(total_blocks);
    break;          case 'd':
                printf("Exiting...\n");

                break;
    default:
                printf("Invalid choice. Please try again.\n");
        }
      } while (choice != 'd');

      return 0;
  }
```

[15]

Q.2 Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

55, 58, 39, 18, 90, 160, 150, 38, 184

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Start Head Position: 50

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

// Function to find the index of the closest request to the current head position
int find_closest_request(int requests[], int num_requests, int current_head) {
int min_distance = INT_MAX;
   int min_index = -1;

   for (int i = 0; i < num_requests; i++) {         int
distance = abs(requests[i] - current_head);         if
(distance < min_distance) {             min_distance
= distance;            min_index = i;
        }
    }

   return min_index;
}

// Function to simulate SSTF disk scheduling algorithm void
sstf(int requests[], int num_requests, int start) {
   int total_head_movements = 0;
int current_head_position = start;

   printf("Order of service:\n");

   for (int i = 0; i < num_requests; i++) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        int closest_index = find_closest_request(requests, num_requests,
current_head_position);
        printf("%d ", requests[closest_index]);
        total_head_movements += abs(requests[closest_index] - current_head_position);
current_head_position = requests[closest_index];

        // Remove the processed request from the array
for (int j = closest_index; j < num_requests - 1; j++) {
requests[j] = requests[j + 1];
        }
        num_requests--;
    }

    printf("\nTotal head movements: %d\n", total_head_movements);

}

int main() {     int requests[] = {55, 58, 39, 18, 90, 160,
150, 38, 184};     int num_requests = sizeof(requests) /
sizeof(requests[0]);     int start = 50;

    printf("Order of service:\n");
    sstf(requests, num_requests, start);

    return 0; }
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 15

Q.1 Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Exit [15]

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_BLOCKS 1000

// Node structure to represent a block in the disk
struct Node {     int block_number;     struct
Node* next;
};

// Global variables
struct Node* disk_blocks[MAX_BLOCKS]; // Array of pointers to nodes representing disk
blocks
int total_blocks; // Total number of disk blocks
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
// Function to initialize the disk with free blocks void
initialize_disk() {
   for (int i = 0; i < total_blocks; i++) {
      disk_blocks[i] = NULL; // Set all pointers to NULL indicating free blocks
   }
}


// Function to randomly mark some blocks as allocated void
random_allocation() {
   srand(time(NULL)); // Seed the random number generator
   int num_allocated = rand() % (total_blocks + 1); // Randomly select the number of
allocated blocks

   printf("Randomly allocated blocks: ");     for (int i = 0; i <
num_allocated; i++) {         int block = rand() % total_blocks; //
Randomly select a block         if (disk_blocks[block] == NULL) {
// If the block is free
         disk_blocks[block] = (struct Node*)malloc(sizeof(struct Node)); // Allocate memory
for the node
         disk_blocks[block]->block_number = block; // Assign block number to the node
disk_blocks[block]->next = NULL; // Set next pointer to NULL            printf("%d ",
block);
      } else { // If the block is already allocated, try another block
i--;
      }
}
   printf("\n");
}

// Function to display the bit vector
void show_bit_vector() {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   printf("Bit Vector (0 indicates free block, 1 indicates allocated block):\n");
for (int i = 0; i < total_blocks; i++) {
       printf("%d ", disk_blocks[i] != NULL ? 1 : 0);
   }
   printf("\n");

}

// Function to create a new file void
create_new_file() {
   printf("Enter the starting block number for the new file: ");
int start_block;     scanf("%d", &start_block);
   if (start_block >= 0 && start_block < total_blocks && disk_blocks[start_block] ==
NULL) {
       disk_blocks[start_block] = (struct Node*)malloc(sizeof(struct Node)); // Allocate
memory for the node
       disk_blocks[start_block]->block_number = start_block; // Assign block number to the
node
       disk_blocks[start_block]->next = NULL; // Set next pointer to NULL
printf("New file created successfully.\n");
   } else {
       printf("Invalid starting block or block already allocated.\n");
   }
}

// Function to display the directory
void show_directory() {
printf("Directory (Allocated blocks):\n");
printf("Start Block\tEnd Block\n");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   int start_block = -1;
int end_block = -1;
   for (int i = 0; i < total_blocks; i++) {         if
(disk_blocks[i] != NULL) { // If the block is allocated
if (start_block == -1) { // If this is the start of a file
start_block = i;

              end_block = i;
           } else { // If this block is part of the file
end_block = i;
           }
        } else { // If the block is free            if
(start_block != -1) { // If the file has ended
printf("%d\t\t%d\n", start_block, end_block);
              start_block = -1;
              end_block = -1;
           }
        }
   }
   if (start_block != -1) { // If the last file extends to the end
printf("%d\t\t%d\n", start_block, end_block);
   }
}

// Function to free memory allocated for files void
free_memory() {
   for (int i = 0; i < total_blocks; i++) {
if (disk_blocks[i] != NULL) {
           free(disk_blocks[i]); // Free memory for each allocated node
disk_blocks[i] = NULL; // Set pointer to NULL
        }
   }
}
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
int main() {
   printf("Enter the total number of disk blocks: ");

   scanf("%d", &total_blocks);

   initialize_disk(); // Initialize the disk with free blocks

   random_allocation(); // Randomly mark some blocks as allocated

   char choice;
do {
      printf("\nMenu:\n");
printf("a) Show Bit Vector\n");
printf("b) Create New File\n");
printf("c) Show Directory\n");
printf("d) Exit\n");
printf("Enter your choice: ");
      scanf(" %c", &choice);

      switch (choice) {
case 'a':
            show_bit_vector();
            break;
case 'b':
create_new_file();
            break;
case 'c':
            show_directory();
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            break;
case 'd':
            printf("Exiting...\n");
            free_memory(); // Free memory allocated for files
break;

default:
            printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 'd');

    return 0;
  }
```

Q.2 Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.. [15]

80, 150, 60,135, 40, 35, 170
Starting Head Position: 70
Direction: Right

```
#include <stdio.h>

#include <stdlib.h>


// Function to sort disk requests in ascending order void

sort_requests(int requests[], int num_requests) {     int

i, j, temp;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    for (i = 0; i < num_requests - 1; i++) {

        for (j = 0; j < num_requests - i - 1; j++) {
if (requests[j] > requests[j + 1]) {
temp = requests[j];              requests[j] =
requests[j + 1];              requests[j + 1] =
temp;
            }
        }
    }
}

// Function to simulate C-SCAN disk scheduling algorithm void
c_scan(int requests[], int num_requests, int start, char direction) {
int total_head_movements = 0;     int current_head_position = start;

    // Sort requests
    sort_requests(requests, num_requests);

    printf("Order of service:\n");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    if (direction == 'R') {
// Move rightwards
        int i;
        for (i = 0; i < num_requests; i++) {
if (requests[i] >= start) {
break;
            }
        }
        for (; i < num_requests; i++) {
printf("%d ", requests[i]);
            total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
        }
        total_head_movements += abs(199 - current_head_position); // Move to the
end        current_head_position = 199; // Update current head position        for (i =
0; i < num_requests; i++) {            if (requests[i] < start) {                printf("%d ",
requests[i]);
                total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
            }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        }
    } else if (direction == 'L') {
        // Move leftwards
        int i;
        for (i = num_requests - 1; i >= 0; i--) {
if (requests[i] <= start) {                break;
            }
}
        for (; i >= 0; i--) {
printf("%d ", requests[i]);
            total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
        }
        total_head_movements += abs(0 - current_head_position); // Move to the
beginning
        current_head_position = 0; // Update current head position
for (i = num_requests - 1; i >= 0; i--) {

            if (requests[i] > start) {
printf("%d ", requests[i]);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
                total_head_movements += abs(requests[i] - current_head_position);
        current_head_position = requests[i];
            }
        }
    }

    printf("\nTotal head movements: %d\n", total_head_movements);
}

int main() {
    int requests[] = {80, 150, 60, 135, 40, 35, 170};     int
num_requests = sizeof(requests) / sizeof(requests[0]);
int start_head_position = 70;     char direction = 'R'; //
Right direction

    printf("Order of service:\n");
    c_scan(requests, num_requests, start_head_position, direction);

    return 0;
}
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.3. Oral/Viva [05]

SLIP 16

Q.1 Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option

- Show Bit Vector
- Create New File
- Show Directory
- Exit

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_BLOCKS 1000


int disk_blocks[MAX_BLOCKS] = {0}; // 0 indicates free block, 1 indicates allocated
block
int total_blocks; // Total number of disk blocks

// Function to randomly mark some blocks as allocated void
random_allocation() {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    srand(time(NULL)); // Seed the random number generator
    int num_allocated = rand() % (total_blocks + 1); // Randomly select the number of
allocated blocks

    printf("Randomly allocated blocks: ");
for (int i = 0; i < num_allocated; i++) {
        int block = rand() % total_blocks; // Randomly select a block
if (disk_blocks[block] == 0) { // If the block is free
disk_blocks[block] = 1; // Mark the block as allocated
printf("%d ", block);
        } else { // If the block is already allocated, try another block
i--;
        }
}
    printf("\n");
}

// Function to display the bit vector
void show_bit_vector() {
    printf("Bit Vector (0 indicates free block, 1 indicates allocated block):\n");
for (int i = 0; i < total_blocks; i++) {
        printf("%d ", disk_blocks[i]);
    }

    printf("\n");
}

// Function to create a new
file void create_new_file() {
int start_block, length;
    printf("Enter the starting block number for the new file:
");     scanf("%d", &start_block);     printf("Enter the length
of the file: ");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    scanf("%d", &length);

    int end_block = start_block + length - 1;

    if (start_block >= 0 && start_block < total_blocks && end_block < total_blocks) {
int i;
        for (i = start_block; i <= end_block; i++) {            if
(disk_blocks[i] == 1) { // If any block is already allocated
printf("Error: Some blocks are already allocated.\n");
return;
            }
        }
        // All blocks are free, so allocate them
for (i = start_block; i <= end_block; i++) {
            disk_blocks[i] = 1;
        }
        printf("New file created successfully.\n");
    } else {
        printf("Error: Invalid starting block or file length.\n");
}
}


// Function to display the directory void
show_directory() {     printf("Directory
(Allocated blocks):\n");     printf("Start
Block\tEnd Block\n");
    int start_block = -1;
int end_block = -1;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    for (int i = 0; i < total_blocks; i++) {        if
(disk_blocks[i] == 1) { // If the block is allocated
if (start_block == -1) { // If this is the start of a file
start_block = i;                end_block = i;
            } else { // If this block is part of the file
end_block = i;
            }
        } else { // If the block is free            if
(start_block != -1) { // If the file has ended
printf("%d\t\t%d\n", start_block, end_block);
                start_block = -1;
                end_block = -1;
            }
        }
    }
    if (start_block != -1) { // If the last file extends to the end
        printf("%d\t\t%d\n", start_block, end_block);
    }
}

int main() {
    printf("Enter the total number of disk blocks: ");     scanf("%d", &total_blocks);

    random_allocation(); // Randomly mark some blocks as allocated

    char choice;
do {
        printf("\nMenu:\n");
printf("a) Show Bit Vector\n");
printf("b) Create New File\n");
printf("c) Show Directory\n");
printf("d) Exit\n");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
    printf("Enter your choice: ");
    scanf(" %c", &choice);

        switch (choice) {
    case 'a':
            show_bit_vector();
            break;
    case 'b':
    create_new_file();
            break;
    case 'c':
            show_directory();
            break;          case
    'd':
    printf("Exiting...\n");
            break;
    default:
            printf("Invalid choice. Please try again.\n");
        }
      } while (choice != 'd');


      return 0;
  }
```

[15]

Q.2 Write an MPI program to find the min number from randomly generated 1000 numbers (stored in array) on a cluster (Hint: Use MPI_Reduce)

```
#include <stdio.h>
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
#include <stdlib.h>

#include <mpi.h>


#define ARRAY_SIZE 1000


int main(int argc, char **argv) {

int rank, size;

   int local_array[ARRAY_SIZE];     int

local_min = INT_MAX, global_min;


   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);


   // Seed the random number generator differently for each process

srand(rank + 1);


   // Generate random numbers for the local array

for (int i = 0; i < ARRAY_SIZE; i++) {

local_array[i] = rand();

   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   // Find the minimum number in the local array
for (int i = 0; i < ARRAY_SIZE; i++) {        if
(local_array[i] < local_min) {            local_min
= local_array[i];
        }
    }


   // Reduce all local minimums to find the global minimum
   MPI_Reduce(&local_min, &global_min, 1, MPI_INT, MPI_MIN, 0,
MPI_COMM_WORLD);



   // Process 0 prints the global minimum
if (rank == 0) {
        printf("Global minimum number: %d\n", global_min);
    }


   MPI_Finalize();
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)
CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
  return 0;

}
```
[15]

Q.3. Oral/Viva [05]

SLIP 18

Q.1 Write a program to simulate Indexed file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned above and implement each option.

- Show Bit Vector
- Create New File
- Show Directory
- Delete File
- Exit

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>


#define MAX_BLOCKS 1000
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)
CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

// Structure to represent a file struct File { int index_block; // Index block pointing to the allocated blocks int length; // Length of the file
};

int disk_blocks[MAX_BLOCKS] = {0}; // 0 indicates free block, 1 indicates allocated block struct File directory[MAX_BLOCKS]; // Directory to store file information int total_blocks; // Total number of disk blocks int num_files = 0; // Number of files in the directory

// Function to randomly mark some blocks as allocated

void random_allocation() { srand(time(NULL)); // Seed the random number generator int num_allocated = rand() % (total_blocks + 1); // Randomly select the number of allocated blocks

printf("Randomly allocated blocks: "); for (int i = 0; i < num_allocated; i++) { int block = rand() % total_blocks; // Randomly select a block if (disk_blocks[block] == 0) { // If the block is free disk_blocks[block] = 1; // Mark the block as allocated printf("%d ", block);
} else { // If the block is already allocated, try another block
i--;

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        }     }
    printf("\n");
    }

    // Function to display the bit vector void show_bit_vector() {     printf("Bit
    Vector (0 indicates free block, 1 indicates allocated block):\n");     for (int i =
    0; i < total_blocks; i++) {

        printf("%d ", disk_blocks[i]);
      }
    printf("\n");
    }

    // Function to create a new file void create_new_file() {
    int index_block, length;     printf("Enter the index block
    number for the new file: ");     scanf("%d", &index_block);
    printf("Enter the length of the file: ");     scanf("%d",
    &length);

      // Check if the index block is valid and not already allocated
      if (index_block >= 0 && index_block < total_blocks
     && disk_blocks[index_block] == 0) {
        // Allocate blocks for the file
    directory[num_files].index_block = index_block;
    directory[num_files].length = length;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
num_files++;        printf("New file created
successfully.\n");
   } else {          printf("Error: Invalid index block or index block already
allocated.\n");     }
}

// Function to display the directory void show_directory() {
printf("Directory (Allocated files):\n");     printf("Index
Block\tLength\n");     for (int i = 0; i < num_files; i++) {
printf("%d\t\t%d\n", directory[i].index_block, directory[i].length);
   }
}

// Function to delete a file void delete_file() {     int
index_block;     printf("Enter the index block number of the
file to delete: ");     scanf("%d", &index_block);

   // Find the file in the directory
int i;
   for (i = 0; i < num_files; i++) {         if
(directory[i].index_block == index_block) {
break;
      }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    }

    // If file is found, mark the blocks as free     if (i < num_files) {
for (int j = index_block; j < index_block + directory[i].length; j++) {
disk_blocks[j] = 0;
        }
        // Remove the file from the directory by shifting
elements        for (int j = i; j < num_files - 1; j++) {
directory[j] = directory[j + 1];
        }        num_files--;        printf("File
deleted successfully.\n");
    } else {        printf("Error: File not
found.\n");
    }
}

int main() {     printf("Enter the total number of
disk blocks: ");     scanf("%d", &total_blocks);
random_allocation(); // Randomly mark some
blocks as allocated

    char choice;     do {
printf("\nMenu:\n");        printf("a)
Show Bit Vector\n");
printf("b) Create New File\n");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
printf("c) Show Directory\n");
printf("d) Delete File\n");
printf("e) Exit\n");
printf("Enter your choice: ");
scanf(" %c", &choice);

    switch (choice) {
case 'a':

show_bit_vector();
break;          case 'b':
create_new_file();
break;          case 'c':
show_directory();
break;          case 'd':
delete_file();
break;          case 'e':
printf("Exiting...\n");
break;          default:
            printf("Invalid choice. Please try again.\n");
    }
  } while (choice != 'e');
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        return 0;
}
```
[15]

Q.2 Write a simulation program for disk scheduling using SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

33, 99, 142, 52, 197, 79, 46, 65 Start
Head Position: 72
Direction: Right

```
#include <stdio.h>

#include <stdlib.h>

// Function to sort disk requests in ascending order
void sort_requests(int requests[], int num_requests) {
int i, j, temp;
   for (i = 0; i < num_requests - 1; i++) {
for (j = 0; j < num_requests - i - 1; j++) {
if (requests[j] > requests[j + 1]) {
temp = requests[j];              requests[j] =
requests[j + 1];
            requests[j + 1] = temp;
         }
      }
   }
}
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
// Function to simulate SCAN disk scheduling algorithm void
scan(int requests[], int num_requests, int start, char direction) {
int total_head_movements = 0;
   int current_head_position = start;

   // Sort requests
   sort_requests(requests, num_requests);

   printf("Order of service:\n");

   // Move rightwards
   if (direction == 'R') {
      int i;

      for (i = 0; i < num_requests; i++) {
if (requests[i] >= start) {
            break;
         }
      }
      for (; i < num_requests; i++) {
printf("%d ", requests[i]);
         total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
      }
      // Move to the end
      total_head_movements += abs(199 - current_head_position);
current_head_position = 199; // Update current head position
      for (i = num_requests - 1; i >= 0; i--) {
         if (requests[i] < start) {
printf("%d ", requests[i]);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
        }
    }
  }
  // Move leftwards
  else if (direction == 'L') {
    int i;
    for (i = num_requests - 1; i >= 0; i--) {
        if (requests[i] <= start) {
            break;
        }
}
    for (; i >= 0; i--) {
printf("%d ", requests[i]);
total_head_movements +=
abs(requests[i] -
current_head_position);
current_head_position =
requests[i];
    }
    // Move to the beginning        total_head_movements +=
abs(0 - current_head_position);        current_head_position =
0; // Update current head position        for (i = 0; i <
num_requests; i++) {            if (requests[i] > start) {
printf("%d ", requests[i]);
            total_head_movements += abs(requests[i] - current_head_position);
current_head_position = requests[i];
        }
    }
  }

  printf("\nTotal head movements: %d\n", total_head_movements); }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
int main() {     int requests[] = {33, 99, 142, 52, 197, 79,
46, 65};     int num_requests = sizeof(requests) /
sizeof(requests[0]);     int start_head_position = 72;     char
direction = 'R'; // Right direction

   printf("Order of service:\n");
   scan(requests, num_requests, start_head_position, direction);

   return 0;
  }       [15]
```

Q.3. Oral/Viva [05]

SLIP 19

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	3	2	4	6	5	4	4	3	4	4	2
P1	1	2	0	1	4	4	4	4				
P2	0	0	0	0	0	0	1	2				
P3	3	3	2	2	3	9	3	4				

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

P4	1	4	3	2	2	5	3	3				
P5	2	4	1	4	4	6	3	4				

a) Calculate and display the content of need matrix?

b) Is the system in safe state? If display the safe sequence.

```
#include <stdio.h>
#include <stdbool.h>

#define P 6 // Number of processes
#define R 4 // Number of resources

void calculateNeedMatrix(int need[P][R], int max[P][R], int allocation[P][R], int
available[R]) {
   for (int i = 0; i < P; i++) {

      for (int j = 0; j < R; j++) {
         need[i][j] = max[i][j] - allocation[i][j];
      }
   }
}

bool isSafeState(int processes[], int available[], int max[][R], int allocation[][R]) {
int need[P][R];
   calculateNeedMatrix(need, max, allocation, available);

   bool finish[P] = {false};
int safeSeq[P];     int
work[R];     for (int i = 0; i <
R; i++) {
      work[i] = available[i];
   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   int count = 0;     while (count
< P) {         bool found = false;
for (int p = 0; p < P; p++) {
          if (!finish[p]) {
             int j;
             for (j = 0; j < R; j++) {
if (need[p][j] > work[j])
                    break;
             }
             if (j == R) {
                for (int k = 0; k < R; k++) {

                    work[k] += allocation[p][k];
                }
                safeSeq[count++] = p;
finish[p] = true;                    found
= true;
             }
          }        }
if (!found) {
          printf("System is not in safe state\n");
          return false;
       }
   }

   printf("System is in safe state.\nSafe sequence is: ");
   for (int i = 0; i < P; i++) {
      printf("P%d ", safeSeq[i]);
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
    }
printf("\n");
return true;
}

int main() {     int processes[] = {0,
1, 2, 3, 4, 5};     int available[] = {3,
4, 4, 2};     int max[][R] = {
        {6, 5, 4, 4},
        {4, 4, 4, 4},
{0, 0, 1, 2},
        {3, 9, 3, 4},

        {2, 5, 3, 3},
        {4, 6, 3, 4}
    };
    int allocation[][R] = {
        {0, 3, 2, 4},
        {1, 2, 0, 1},
{0, 0, 0, 0},
        {3, 3, 2, 2},
        {1, 4, 3, 2},
        {2, 4, 1, 4}
    };

    isSafeState(processes, available, max, allocation);

    return 0;
  }
```

[15]

Q.2 Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

the list of request in the order in which it is served. Also display the total number of head moments.

23, 89, 132, 42, 187, 69, 36, 55

Start Head Position: 40 Direction:

Left

```
#include <stdio.h>
#include <stdlib.h>

void sort(int arr[], int n) {
for (int i = 0; i < n - 1; i++) {
for (int j = 0; j < n - i - 1; j++)
{            if (arr[j] > arr[j + 1])
{                int temp = arr[j];
arr[j] = arr[j + 1];
              arr[j + 1] = temp;
           }
        }
    }
}

void c_scan(int disk[], int n, int start, char direction) {
int head_movements = 0;
    int end = 199; // Total number of disk blocks

    if (direction == 'L') {         for (int i = start; i
>= 0; i--) {             printf("%d ", disk[i]);
head_movements += abs(disk[i] - start);
           start = disk[i];
        }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        head_movements += start; // Move to the left end
printf("0 ");
        for (int i = end; i >= start; i--) {
printf("%d ", i);            head_movements
+= abs(i - start);            start = i;
        }
    } else if (direction == 'R') {         for (int i =
start; i <= end; i++) {            printf("%d ",
disk[i]);            head_movements +=
abs(disk[i] - start);            start = disk[i];
        }
        head_movements += end - start; // Move to the right end
printf("%d ", end);         for (int i = 0; i <= start; i++) {
printf("%d ", i);
            head_movements += abs(i - start);
start = i;
        }
    }

    printf("\nTotal number of head movements: %d\n", head_movements); }

int main() {
    int disk[] = {23, 89, 132, 42, 187, 69, 36,
55};     int n = sizeof(disk) / sizeof(disk[0]);
int start_head_position = 40;
    char direction = 'L'; // Left direction as given

    sort(disk, n);
    printf("Order of service: ");
    c_scan(disk, n, start_head_position, direction);

    return 0;
  }
```

[15]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.3. Oral/Viva [05] SLIP 20

Q.1 Write a simulation program for disk scheduling using SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.
33, 99, 142, 52, 197, 79, 46, 65
Start Head Position: 72
Direction: User defined

```
#include <stdio.h>
#include <stdlib.h>

void sort(int arr[], int n) {     for (int
i = 0; i < n - 1; i++) {         for (int j
= 0; j < n - i - 1; j++) {            if
(arr[j] > arr[j + 1]) {               int
temp = arr[j];               arr[j] = arr[j
+ 1];
            arr[j + 1] = temp;
          }
       }
    }
}

void scan(int disk[], int n, int start, char direction) {
int head_movements = 0;
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   int end = 199; // Total number of disk blocks

   if (direction == 'L') {
printf("Order of service: ");        for
(int i = start; i >= 0; i--) {
printf("%d ", disk[i]);
          head_movements += abs(disk[i] - start);
          start = disk[i];
       }
       head_movements += start; // Move to the left
end        printf("0 ");        for (int i = 0; i < n; i++) {
if (disk[i] > start && disk[i] <= end) {
printf("%d ", disk[i]);
             head_movements += abs(disk[i] - start);
             start = disk[i];
          }
       }
   } else if (direction == 'R') {
printf("Order of service: ");        for
(int i = start; i <= end; i++) {
printf("%d ", disk[i]);
          head_movements += abs(disk[i] - start);
          start = disk[i];
       }
       head_movements += end - start; // Move to the right end
printf("%d ", end);        for (int i = n - 1; i >= 0; i--) {
if (disk[i] < start && disk[i] >= 0) {

             printf("%d ", disk[i]);
             head_movements += abs(disk[i] - start);
start = disk[i];
          }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
        }
    }

    printf("\nTotal number of head movements: %d\n", head_movements); }

int main() {
    int disk[] = {33, 99, 142, 52, 197, 79, 46,
65};     int n = sizeof(disk) / sizeof(disk[0]);
int start_head_position = 72;     char direction;

    printf("Enter direction (L for left, R for right): ");
scanf(" %c", &direction);

    sort(disk, n);
    scan(disk, n, start_head_position, direction);

    return 0;
  }
```

[15]

Q.2 Write an MPI program to find the max number from randomly generated 1000 numbers (stored in array) on a cluster (Hint: Use MPI_Reduce)

```
#include <stdio.h>

#include <stdlib.h>

#include <mpi.h>


#define ARRAY_SIZE 1000
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
int main(int argc, char *argv[]) {
int rank, size;
   int data[ARRAY_SIZE];
int local_max, global_max;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   // Seed random number generator
srand(rank);

   // Generate random numbers     for (int
i = 0; i < ARRAY_SIZE; i++) {
data[i] = rand();
   }

   // Find local maximum
local_max = data[0];
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   for (int i = 1; i < ARRAY_SIZE; i++) {
if (data[i] > local_max) {
local_max = data[i];
      }
   }

   // Find global maximum using MPI_Reduce
   MPI_Reduce(&local_max, &global_max, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);

   if (rank == 0) {
      printf("Global maximum: %d\n", global_max);
   }

   MPI_Finalize();

   return 0;
  }
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 21

Q.1 Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

55, 58, 39, 18, 90, 160, 150, 38, 184

Start Head Position: 50

```
 #include <stdio.h>
#include <stdlib.h>

void fcfs(int disk[], int n, int start) {
int head_movements = 0;

   printf("Order of service: ");
   printf("%d ", start); // Start head position

   // Iterate through disk requests
for (int i = 0; i < n; i++) {
printf("%d ", disk[i]); // Print the
request being serviced
head_movements += abs(disk[i] -
start); // Calculate head movements
start = disk[i]; // Move head to the
current request
   }

   printf("\nTotal number of head movements: %d\n", head_movements); }

int main() {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   int disk[] = {55, 58, 39, 18, 90, 160, 150, 38, 184};
int n = sizeof(disk) / sizeof(disk[0]);
   int start_head_position = 50;

   printf("Order of service: ");
   fcfs(disk, n, start_head_position);

   return 0;
  }
```

Q.2 Write an MPI program to calculate sum of all even randomly generated 1000 numbers (stored in array) on a cluster

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000

int main(int argc, char *argv[]) {
int rank, size;

   int data[ARRAY_SIZE];     int
local_sum = 0, global_sum = 0;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   // Seed random number generator
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
   srand(rank);

   // Generate random numbers     for (int
i = 0; i < ARRAY_SIZE; i++) {
       data[i] = rand();
   }

   // Calculate local sum of even numbers
for (int i = 0; i < ARRAY_SIZE; i++) {
       if (data[i] % 2 == 0) {
           local_sum += data[i];
       }
   }

   // Calculate global sum of even numbers using MPI_Reduce
   MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

   if (rank == 0) {
       printf("Global sum of even numbers: %d\n", global_sum);
   }


   MPI_Finalize();

   return 0;
}
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

SLIP 22

Q.1 Write an MPI program to calculate sum of all odd randomly generated 1000 numbers (stored in array) on a cluste

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000

int main(int argc, char *argv[]) {
   int rank, size;
   int data[ARRAY_SIZE];
   int local_sum = 0, global_sum = 0;

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

   // Seed random number generator
   srand(rank);

   // Generate random numbers     for (int
i = 0; i < ARRAY_SIZE; i++) {
       data[i] = rand();
   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   // Calculate local sum of odd numbers
for (int i = 0; i < ARRAY_SIZE; i++) {
       if (data[i] % 2 != 0) {
           local_sum += data[i];
       }
   }

   // Calculate global sum of odd numbers using MPI_Reduce
   MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

   if (rank == 0) {
       printf("Global sum of odd numbers: %d\n", global_sum);
   }

   MPI_Finalize();

   return 0;
  }.
```
[15]

Q.2 Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option

- Show Bit Vector
- Delete already created file
- Exit [15]

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)
CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
void initializeBitVector(int bitVector[], int n) {
   // Initialize all blocks as free (0)
for (int i = 0; i < n; i++) {
bitVector[i] = 0;
   }
}

void markBlocksAsAllocated(int bitVector[], int n) {
// Randomly mark some blocks as allocated (1)
srand(time(NULL));
   int numAllocatedBlocks = rand() % n + 1; // Random number of allocated blocks
for (int i = 0; i < numAllocatedBlocks; i++) {
      int block = rand() % n;
if (bitVector[block] == 0) {
         bitVector[block] = 1;
      }
   }

}

void showBitVector(int bitVector[], int n) {
printf("Bit Vector (0 = Free, 1 = Allocated):\n");
   for (int i = 0; i < n; i++) {
      printf("%d ", bitVector[i]);
   }
   printf("\n");
}

void deleteFile(int bitVector[], int n) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    int block;
    printf("Enter block number to delete file: ");
    scanf("%d", &block);

    if (block >= 0 && block < n) {
if (bitVector[block] == 1) {
bitVector[block] = 0;
            printf("File deleted successfully from block %d.\n", block);
        } else {
            printf("Block %d is not allocated.\n", block);
        }
    } else {
        printf("Invalid block number.\n");
    }
}

int    main()    {
int              n;
printf("Enter
the  number  of
blocks   in   the
disk:          ");
scanf("%d",
&n);

    int bitVector[n]; // Bit vector to represent disk blocks

    initializeBitVector(bitVector, n);
    markBlocksAsAllocated(bitVector, n);

    int choice;     do {
printf("\nMenu:\n");        printf("1. Show Bit
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
 Vector\n");        printf("2. Delete already
 created file\n");
        printf("3. Exit\n");
 printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
 case 1:
                showBitVector(bitVector, n);
 break;            case 2:
 deleteFile(bitVector, n);
                break;            case
 3:
 printf("Exiting...\n");
                break;
 default:
                printf("Invalid choice. Please try again.\n");
        }

    } while (choice != 3);

    return 0;
 }
```

Q.3. Oral/Viva [05]

SLIP 24

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.1 Write an MPI program to calculate sum of all odd randomly generated 1000 numbers
(stored in array) on a cluster.

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000

int main(int argc, char *argv[]) {
   int rank, size;
   int data[ARRAY_SIZE];
   int local_sum = 0, global_sum = 0;

   MPI_Init(&argc, &argv);

   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   // Seed random number generator
   srand(rank);

   // Generate random numbers     for (int
i = 0; i < ARRAY_SIZE; i++) {
       data[i] = rand();
   }

   // Calculate local sum of odd numbers
for (int i = 0; i < ARRAY_SIZE; i++) {
       if (data[i] % 2 != 0) {
           local_sum += data[i];
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
        }
    }

    // Calculate global sum of odd numbers using MPI_Reduce
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Global sum of odd numbers: %d\n", global_sum);
    }

    MPI_Finalize();

    return 0;
}
```

[15]

Q.2 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. The following snapshot of system, A, B, C and D are the resource type.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

a) Calculate and display the content of need matrix?
b) Is the system in safe state? If display the safe sequence.

```
#include <stdio.h>
#include <stdbool.h>

#define P 5 // Number of processes
#define R 3 // Number of resources

void calculateNeedMatrix(int need[P][R], int max[P][R], int allocation[P][R], int
available[R]) {
   for (int i = 0; i < P; i++) {
for (int j = 0; j < R; j++) {
          need[i][j] = max[i][j] - allocation[i][j];
       }
   }
}

bool isSafeState(int processes[], int available[], int max[][R], int allocation[][R]) {

   int need[P][R];
   calculateNeedMatrix(need, max, allocation, available);

   bool finish[P] = {false};
int safeSeq[P];    int
work[R];    for (int i = 0; i <
R; i++) {        work[i] =
available[i];
   }

   int count = 0;    while (count
< P) {        bool found = false;
for (int p = 0; p < P; p++) {
if (!finish[p]) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            int j;
            for (j = 0; j < R; j++) {
if (need[p][j] > work[j])
                break;
            }            if (j == R) {
for (int k = 0; k < R; k++) {
                work[k] += allocation[p][k];
              }
              safeSeq[count++] = p;
finish[p] = true;                found
= true;
            }
          }       }
if (!found) {
printf("System is
not in safe
state\n");
          return false;
       }
    }

   printf("System is in safe state.\nSafe sequence is: ");
   for (int i = 0; i < P; i++) {
printf("P%d ", safeSeq[i]);
   }
printf("\n");
return true;
}
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
int main() {     int processes[] =
{0, 1, 2, 3, 4};     int available[] =
{0, 0, 0};     int max[][R] = {
        {0, 0, 0},
        {2, 0, 2},
        {0, 0, 0},
        {1, 0, 0},
        {0, 0, 2}
    };
    int allocation[][R] = {
        {0, 1, 0},
        {2, 0, 2},
        {3, 0, 3},
        {2, 1, 1},
        {0, 0, 2}

    };

    isSafeState(processes, available, max, allocation);

    return 0;
}
```

[15]

Q.3. Oral/Viva [05]

SLIP 26

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Q.1 Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type.

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

a) Calculate and display the content of need matrix?

b) Is the system in safe state? If display the safe sequence. [15]

```
 #include <stdio.h>
#include <stdbool.h>

#define P 5 // Number of processes
#define R 4 // Number of resources

void calculateNeedMatrix(int need[P][R], int max[P][R], int allocation[P][R], int
available[R]) {
   for (int i = 0; i < P; i++) {
for (int j = 0; j < R; j++) {
          need[i][j] = max[i][j] - allocation[i][j];
      }
   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
}

bool isSafeState(int processes[], int available[], int max[][R], int allocation[][R]) {
int need[P][R];
   calculateNeedMatrix(need, max, allocation, available);

   bool finish[P] = {false};
int safeSeq[P];     int
work[R];     for (int i = 0; i <
R; i++) {
       work[i] = available[i];
   }

   int count = 0;

   while (count < P) {         bool
found = false;        for (int p =
0; p < P; p++) {             if
(!finish[p]) {
              int j;
              for (j = 0; j < R; j++) {
if (need[p][j] > work[j])
                     break;
              }               if (j == R) {
for (int k = 0; k < R; k++) {
                     work[k] += allocation[p][k];
                  }
                  safeSeq[count++] = p;
finish[p] = true;                    found
= true;
              }
           }         }
if (!found) {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
            printf("System is not in safe state\n");
            return false;
        }
    }

    printf("System is in safe state.\nSafe sequence is: ");
    for (int i = 0; i < P; i++) {
        printf("P%d ", safeSeq[i]);
    }
printf("\n");
return true;
}

int main() {     int processes[] =
{0, 1, 2, 3, 4};     int available[] =
{1, 5, 2, 0};     int max[][R] = {
        {0, 0, 1, 2},
        {1, 7, 5, 0},
        {2, 3, 5, 6},
        {0, 6, 5, 2},
        {0, 6, 5, 6}
    };
    int allocation[][R] = {
        {0, 0, 1, 2},
{1, 0, 0, 0},
        {1, 3, 5, 4},
        {0, 6, 3, 2},
        {0, 0, 1, 4}
    };
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
  isSafeState(processes, available, max, allocation);

  return 0;
}
```

Q.2 Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

56, 59, 40, 19, 91, 161, 151, 39, 185
Start Head Position: 48

```
#include <stdio.h>
#include <stdlib.h>

void fcfs(int disk[], int n, int start) {
int head_movements = 0;

   printf("Order of service: ");
   printf("%d ", start); // Start head position

   // Iterate through disk requests
for (int i = 0; i < n; i++) {
       printf("%d ", disk[i]); // Print the request being serviced
       head_movements += abs(disk[i] - start); // Calculate head movements
start = disk[i]; // Move head to the current request
   }

   printf("\nTotal number of head movements: %d\n", head_movements); }

int main() {
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)
CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
    int disk[] = {56, 59, 40, 19, 91, 161, 151, 39,
 185};     int n = sizeof(disk) / sizeof(disk[0]);     int
 start_head_position = 48;

    printf("Order of service: ");
 fcfs(disk, n, start_head_position);
 return 0; }
```

[15]

Q.3. Oral/Viva [05]

SLIP 30

Q.1 Write an MPI program to find the min number from randomly generated 1000 numbers (stored in array) on a cluster (Hint: Use MPI_Reduce) [15]

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

#define ARRAY_SIZE 1000
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
int main(int argc, char *argv[]) {
int rank, size;
   int data[ARRAY_SIZE];

   int local_min, global_min;


   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);


   // Seed random number generator
srand(rank);


   // Generate random numbers     for (int
i = 0; i < ARRAY_SIZE; i++) {
data[i] = rand();
   }


   // Find local minimum     local_min =
data[0];     for (int i = 1; i <
ARRAY_SIZE; i++) {         if (data[i] <
local_min) {             local_min = data[i];
       }
   }
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October
(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

```
   // Find global minimum using MPI_Reduce
   MPI_Reduce(&local_min, &global_min, 1, MPI_INT, MPI_MIN, 0,
MPI_COMM_WORLD);


   if (rank == 0) {        printf("Global minimum:
%d\n", global_min);
   }


   MPI_Finalize();


   return 0;
 }
```

Q.2 Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.

65, 95, 30, 91, 18, 116, 142, 44, 168

Start Head Position: 52

```
#include <stdio.h>
#include <stdlib.h>

void fcfs(int disk[], int n, int start) {
int head_movements = 0;
printf("Order of service: ");
```


Downloaded by Aashish Godse (aashishgodse406@gmail.com)

```
    printf("%d ", start); // Start head position

    // Iterate through disk requests
 for (int i = 0; i < n; i++) {
        printf("%d ", disk[i]); // Print the request being serviced
 head_movements += abs(disk[i] - start); // Calculate head movements
 start = disk[i]; // Move head to the current request
    }

    printf("\nTotal number of head movements: %d\n", head_movements); }

 int main() {
    int disk[] = {65, 95, 30, 91, 18, 116, 142, 44,
 168};     int n = sizeof(disk) / sizeof(disk[0]);     int
 start_head_position = 52;

    printf("Order of service: ");
    fcfs(disk, n, start_head_position);

    return 0; }
```

[15]

Q.3. Oral/Viva [05]

Downloaded by Aashish Godse (aashishgodse406@gmail.com)

Savitribai Phule Pune University

T.Y.B.Sc.(Computer Science) Practical Examination, March/October

(2019 Pattern)

CS-367 Lab Course-I Operating System-II

Duration: 3 Hrs **Max. Marks: 35**

Downloaded by Aashish Godse (aashishgodse406@gmail.com)